



Docker Interview Questions and Answers – Part 03

What is a Container?

A **container** is a lightweight, portable and executable packages that consist of an application and all its dependencies into a single isolated environment.

Containers allow you to run applications consistently across various computing environments without worrying about compatibility issues between development, staging and production environments.

Containers leverage the host operating system's kernel and are isolated from each other and the host, but they share the OS kernel. This makes them faster and more resource-efficient than traditional virtual machines

Why Containers are Needed? What is Their Goal?

Containers are needed to address challenges in software development, deployment, and scalability:

1. **Consistency Across Environments:** Containers encapsulate an application with all of its dependencies, ensuring that the application runs the same way regardless of where it is deployed. This eliminates the "works on my machine" problem.
2. **Portability:** Once a container is built, it can run on any platform that supports containerization, be it on a developer's laptop, a QA environment, or a cloud infrastructure. This makes it easy to move applications between different environments.
3. **Isolation:** Containers provide application-level isolation, which means you can run multiple containers with different dependencies on the same host without conflict.
4. **Scalability and Efficiency:** Containers are lightweight and efficient, using fewer resources compared to virtual machines. This makes it easier to scale applications, especially in cloud environments where resource optimization is critical.
5. **Faster Deployment:** Containers start much faster than virtual machines, making continuous integration and continuous delivery (CI/CD) pipelines quicker and more efficient.

Goal: The main goal of containers is to provide **faster, more consistent and more scalable deployment** while enabling efficient use of resources across multiple environments and platforms

What is a Container Image?

A **container image** is a static, read-only file that includes the application code, libraries, runtime, environment variables and configuration files necessary for the application to run inside a container. It's essentially a blueprint or template used to create containers.

- **Layers:** A container image is made up of layers, where each layer represents a specific change or addition (such as installing dependencies or adding code). These layers are stacked on top of one another.
- **Storage:** Images are stored in container registries, like Docker Hub, Google Container Registry (GCR), or Amazon Elastic Container Registry (ECR), and can be pulled by the container runtime (e.g., Docker, Kubernetes) when deploying

How Are Containers Different from Virtual Machines (VMs)?

Containers and Virtual Machines (VMs) are both technologies for isolating workloads, but they work in different ways:

Feature	Containers	Virtual Machines (VMs)
Isolation Level	Application-level isolation (same OS kernel)	Hardware-level isolation (full OS and hypervisor)
Resource Overhead	Low, uses less memory and CPU	High, each VM includes a full OS and overhead from the hypervisor
Boot Time	Seconds, containers start very quickly	Minutes, VMs require booting a full OS
Size	Smaller, containers only contain the app and dependencies	Larger, VMs contain a full OS, app, and dependencies
Performance	Faster, uses the host OS kernel directly	Slower, due to virtualization overhead
Portability	High, as long as the host supports containers	Moderate, VMs require specific hypervisors
Use Case	Microservices, DevOps, Continuous Integration/Continuous Deployment (CI/CD)	Legacy applications, applications requiring full OS-level isolation

In summary:

- **Containers** are more lightweight and efficient, ideal for cloud-native applications, microservices, and fast scaling.

VMs are more suitable for running applications that require complete isolation, a full OS, or when you need to support multiple different OS types on the same physical machine.

In Which Scenarios Would You Use Containers and in Which You Would Prefer to Use VMs?

Use Containers When:

1. **Microservices:** When you are working with a microservices architecture, where different services are isolated, and each service can run in its own container.
2. **Cloud-Native Apps:** When your application is designed to be deployed in cloud environments (e.g., Kubernetes), containers provide better portability and flexibility.
3. **Fast Deployment and Scaling:** Containers are quick to start and their lightweight nature makes it easier to scale up or down quickly.
4. **CI/CD Pipelines:** Containers are ideal for continuous integration and delivery (CI/CD), where environments need to be consistent and fast.
5. **Environment Consistency:** Containers provide consistent environments across various stages of the lifecycle (dev, test, prod) without worrying about OS and dependency conflicts.

Use VMs When:

1. **Legacy Applications:** When working with legacy software that requires a specific operating system or needs to run on a separate OS (Windows on Linux, or vice versa).
2. **Full Isolation:** When you require full OS-level isolation for security or regulatory reasons.
3. **Running Multiple OS Types:** If you need to run applications that require different operating systems (e.g., Windows and Linux).
4. **Resource-Heavy Applications:** For applications requiring significant resources or full system control (e.g., databases, big data workloads, or specialized software).
5. **Hardware Virtualization:** If you need to virtualize hardware resources (like CPU, memory, storage) and run multiple OS instances on top of a host OS.

Describe the Process of Containerizing an Application

The process of **containerizing an application** involves packaging the application and its dependencies into a container image. Here's a high-level step-by-step process:

1. **Create a Dockerfile:**
 - A Dockerfile is a script that contains a series of instructions on how to build a container image. This file will typically contain the following steps:
 - **Base Image:** Choose a base image (e.g., `ubuntu`, `node`, `python`) that provides the environment for the app.
 - **Install Dependencies:** Install the necessary packages or libraries.
 - **Add Application Code:** Copy the application code into the container.
 - **Configure Environment Variables:** Set any environment variables or application configurations.
 - **Expose Ports:** Specify which ports the container should expose.
 - **Start the Application:** Define the command that runs the application when the container starts.
2. **Build the Docker Image:**
 - Use the `docker build` command to create a container image from the Dockerfile.

```
docker build -t myapp:latest .
```

3. Test the Image Locally:

- Run the image as a container locally to ensure that the application works as expected.

```
docker run -p 8080:80 myapp:latest
```

4. Push the Image to a Container Registry:

- Push the image to a container registry like Docker Hub, AWS ECR, or Google Container Registry.

```
docker push myapp:latest
```

5. Deploy the Container:

The container image can now be deployed in various environments, including Kubernetes, Docker Swarm, or cloud platforms (AWS, GCP, Azure).

What Are Some of the Advantages of Using Containers?

Here are some key **advantages of using containers**, especially when compared to virtual machines (VMs):

1. Efficiency and Lightweight:

- Containers share the host OS kernel and require less overhead than VMs, which need their own full OS instance.
- Containers are lightweight, using less CPU, memory, and disk space compared to VMs.

2. Fast Startup:

- Containers start almost instantly because they don't need to boot an entire operating system. This enables faster deployments and better scalability.

3. Portability:

- Containers package an application with its dependencies, making it easy to move between different environments (e.g., from developer machines to testing to production).

4. Consistency Across Environments:

- By using containers, you ensure that your application runs the same way regardless of where it is deployed. This eliminates environment discrepancies and reduces deployment issues.

5. Scalability:

- Containers can scale more easily than VMs. Tools like Kubernetes can automatically scale containers based on load, ensuring that resources are used efficiently.

6. Easier CI/CD:

- Containers are ideal for continuous integration/continuous deployment (CI/CD) workflows. They allow for automated testing, building, and deployment in a consistent environment.

7. Cost-Effective:

- Since containers are smaller, they can run more workloads on the same infrastructure compared to VMs, leading to better utilization of resources and cost savings.

8. Version Control and Rollbacks:

- Container images can be versioned and tagged. This makes rolling back to a previous version of an application much easier and more controlled.

How to Run a Container?

To run a container using Docker, you can use the `docker run` command. The basic syntax is:

```
docker run <options> <image-name> <command>
```

For example, to run an **Ubuntu** container, you would use:

```
docker run -it ubuntu
```

Here:

- `-it`: This flag is used to run the container in **interactive mode** (`-i`) and allocate a **pseudo-TTY** (`-t`), which allows you to interact with the container (e.g., access the terminal).
- `ubuntu`: The name of the image you want to run (this would pull the Ubuntu image if not already present locally).

By default, this will run the Ubuntu container and give you an interactive shell within the container. Once inside, you can run commands as you would in any other Linux system.

Why After Running `docker container run ubuntu`, the Output of `docker container ls` is Empty?

The reason `docker container ls` is empty after running the following:

```
docker container run ubuntu
```

is because the container **exited immediately** after it started.

The default behavior when you run a container (like the Ubuntu container in this case) without specifying a command is to execute the default command defined in the container's image (usually some shell or application). If you don't provide an interactive shell (i.e., the container doesn't remain running in the foreground), the container will exit once the default command completes. So, in your case, the container starts and then immediately exits because it doesn't have a running process.

To keep the container running, you can run a command that will keep it alive. For example:

```
docker run -it ubuntu /bin/bash
```

This command will run the container with an interactive Bash shell, which will keep the container running as long as the shell is open.

Alternatively, you could use:

```
docker run -d ubuntu tail -f /dev/null
```

This runs the container in **detached mode** (`-d`) and uses the `tail -f /dev/null` command, which will keep the container alive indefinitely.

How to List All the Containers on the Local Host?

To list all containers on the local Docker host (including both running and stopped containers), you can use the following command:

```
docker ps -a
```

- `docker ps`: Lists only the running containers.
- `-a`: Shows all containers (including those that are stopped or exited).

This will show you a list of containers with their IDs, names, status, and other details.

How to Attach Your Shell to a Terminal of a Running Container?

To attach your shell to a running container (i.e., to interact with the container's terminal), use the `docker exec` command.

Here's the syntax:

```
docker exec -it <container-id> <command>
```

For example, to access the shell of a running container (assuming the container is running an Ubuntu image), you would use:

```
docker exec -it <container-id> /bin/bash
```

- `-it`: This flag ensures you run in interactive mode with a TTY.
- `<container-id>`: The ID or name of the container you want to attach to.
- `/bin/bash`: The command to run within the container (in this case, starting a Bash shell).

To find the container ID or name, you can first list running containers using:

```
docker ps
```

Once you have the container ID or name, you can attach to it using the `docker exec` command as shown above.

If the container is running a shell other than Bash (e.g., `sh`), you can use that instead:

```
docker exec -it <container-id> /bin/sh
```

This will open an interactive shell session inside the running container.

True or False? You Can Remove a Running Container if It Doesn't Run Anything

True.

You can remove a running container even if it doesn't run anything. However, you can't **delete** a container that is running processes directly. If you attempt to remove a running container, you'll encounter an error. To remove a running container, you need to **stop** it first, which will stop any active processes inside the container.

How to Stop and Remove a Container?

To **stop** and **remove** a container, follow these steps:

1. Stop the container:

To stop a running container, use the following command:

```
docker stop <container-id>
```

Here, <container-id> is the container's ID or name.

2. Remove the container:

After stopping the container, you can remove it with:

```
docker rm <container-id>
```

You can also combine the two commands (stop and remove) into one command using the **-f** (force) flag:

```
docker rm -f <container-id>
```

This will stop the container (if running) and then remove it.

What Happens When You Run `docker container run ubuntu`?

When you run:

```
docker container run ubuntu
```

- **Pulling the image:** Docker checks if the **Ubuntu image** is available locally. If not, it will download the image from Docker Hub.
- **Running the container:** Once the image is available, it creates and runs a new container from the Ubuntu image.
- **Default command:** By default, the container will run its entrypoint, which is typically a shell or basic process. Since the Ubuntu image doesn't have an interactive command specified, the container will start, perform the default behavior (likely exit immediately), and then stop.

You won't be able to interact with it unless you specify an interactive command like `/bin/bash`.

How to Run a Container in the Background?

To run a container in the background (detached mode), use the `-d` flag with the `docker run` command. For example:

```
docker run -d ubuntu tail -f /dev/null
```

Explanation:

- `-d`: Runs the container in **detached mode**, meaning it runs in the background.
- `tail -f /dev/null`: A command that will run indefinitely (it's a trick to keep the container alive).

In detached mode, the container will continue running in the background without occupying the terminal. You can check the running containers using:

```
docker ps
```

This will show you the list of running containers, including their status, container IDs, and names.

How to see the processes running inside the container?

You can use the following command to see processes inside a container:

```
docker exec <container-id> ps aux
```

This will list all processes running inside the container, including the `sleep 100` process.

Alternatively, if you want to see the processes directly from the host, you can list the processes running on the host's **docker engine** using:

```
ps aux | grep docker
```

However, note that this will only show the `docker` processes and not the individual processes inside containers unless you're using specific monitoring tools like **cAdvisor** or **Docker stats**.

True or False? If image `httpd-service` has an entry point for running the `httpd` service then, the following will run the container and eventually the `httpd` service `docker run httpd-service ls`

False.

Here's why:

When you run `docker run httpd-service ls`, you're telling **Podman** to run the `httpd-service` image and execute the `ls` command inside the container.

However, the behavior of this command depends on the **entry point** defined in the Dockerfile or image configuration for `httpd-service`.

If the `httpd-service` image has an entry point (via the `ENTRYPOINT` directive in the Dockerfile) set to start the `httpd` service, then `docker run httpd-service` will start the container and execute the `httpd` service (likely running Apache HTTP server inside the container). But when you append `ls`, this will override the entry point because you're explicitly telling Podman to run the `ls` command, which is not related to the container's entry point.

The key points are:

1. **ENTRYPOINT:** If the image is set up to start `httpd` (via `ENTRYPOINT` or `CMD`), that will be the default behavior when the container runs without specifying a command.
2. **Command Override:** If you specify a command (like `ls`), it will override the entry point and the container will run `ls` instead of the `httpd` service. So, **the `httpd` service will not start**, and only `ls` will be executed inside the container.

What you can do:

To run the container and have the `httpd` service start, you should run the container **without specifying `ls`**:

```
docker run httpd-service
```

If you want to explicitly run the `httpd` service while specifying a command, you could do something like:

```
bash
CopyEdit
podman run httpd-service /usr/sbin/httpd -D FOREGROUND
```

In this case, it will run the `httpd` service directly even if the image has an entry point.

True or False? Running `docker restart CONTAINER_NAME` kills the main process inside the container and runs it again from scratch

True, with a clarification.

When you run the command:

```
docker restart CONTAINER_NAME
```

What happens is:

1. **Docker sends a signal to stop the container**, which gracefully shuts down the running container, including killing the main process inside the container.
2. **The container is then restarted**, and Docker starts the container from scratch by running the command specified in the container's entry point or command.

This means the main process inside the container will be killed (if it hasn't already shut down), and it will be restarted from its initial entry point, which is usually the main service or command that was specified when the container was created.

Key Points:

- **Graceful Stop:** By default, Docker will attempt a graceful shutdown of the container, but after a timeout, it forcefully kills the container if it's still running.
- **Start from Scratch:** After stopping, the container is restarted, and the entry point (or command) is executed again. This is effectively "running it from scratch."

So, **yes**, the main process will be terminated and started again, as part of the restart process.

You would like to run a web server inside a container but, be able to access it from the localhost. Demonstrate how to do that

To run a web server inside a Docker container and access it from your localhost, you need to follow these steps. For this example, we'll use the **nginx** web server container as a demonstration.

Steps:

1. Run a web server (nginx) container with port mapping:

The idea is to map a port on your **localhost** to a port inside the container. The web server will be accessible via the mapped port on the host machine.

You can use the following command to run the **nginx** web server container:

```
docker run -d -p 8080:80 --name my-web-server nginx
```

Explanation:

- **-d:** Run the container in detached mode (in the background).
- **-p 8080:80:** Maps port 8080 on your **localhost** (host machine) to port 80 inside the container, where the nginx web server is running.
- **--name my-web-server:** Assigns a name (my-web-server) to the container for easier management.
- **nginx:** The Docker image for the nginx web server.

2. Access the web server from your localhost:

After running the above command, the web server inside the container is accessible from your host machine's **localhost** via port 8080.

Open a web browser and visit:

```
http://localhost:8080
```

You should see the default nginx welcome page, which confirms the web server is running inside the container and accessible from your localhost.

3. Verify the container is running:

You can check the running containers with:

```
docker ps
```

This should show the `my-web-server` container with port 8080 on your local machine mapped to port 80 inside the container.

4. Stop and remove the container (if necessary):

If you want to stop the container, you can use:

```
docker stop my-web-server
```

To remove the container after stopping it:

```
docker rm my-web-server
```

Notes:

- **Port Mapping** (`-p host_port:container_port`) is essential to allow communication between your localhost and the container.

You can replace **8080** with any other available port on your host if needed.

After running a container, it stopped. `docker ps` shows nothing. How can you show its details?

When you run a Docker container and it stops, it will not show up in the output of `docker ps` (because `docker ps` only lists running containers). However, you can view the stopped container and its details by using the `docker ps -a` command, which lists all containers, including those that have stopped.

Steps to view details of a stopped container:

1. List all containers (including stopped ones):

Run the following command to list all containers:

```
docker ps -a
```

This will show both running and stopped containers, along with their container IDs, names, status, and other relevant information.

2. View the logs of the stopped container:

If you want to check the logs of the stopped container (to understand why it stopped), use:

```
docker logs <CONTAINER_ID or CONTAINER_NAME>
```

Replace `<CONTAINER_ID or CONTAINER_NAME>` with the actual container ID or name, which you can get from the `docker ps -a` output.

For example:

```
docker logs my-web-server
```

3. Inspect the container's details:

To view detailed information about a stopped container (such as its configuration, environment variables, mount points, etc.), use:

```
docker inspect <CONTAINER_ID or CONTAINER_NAME>
```

This will return a JSON output with detailed information about the container.

For example:

```
docker inspect my-web-server
```

4. Restart the stopped container (if needed):

If you want to restart the container, you can use:

```
docker start <CONTAINER_ID or CONTAINER_NAME>
```

Example:

```
docker start my-web-server
```

After starting it, if you want to attach your terminal to the running container, you can use:

```
docker attach <CONTAINER_ID or CONTAINER_NAME>
```

5. Check the exit code of the container:

To see the exit status (exit code) of the stopped container (which might help you understand why it stopped), use:

```
docker inspect --format '{{.State.ExitCode}}' <CONTAINER_ID or CONTAINER_NAME>
```

Example:

```
docker inspect --format '{{.State.ExitCode}}' my-web-server
```

The exit code 0 typically means the container exited successfully, while non-zero values indicate errors.

Summary:

- Use `docker ps -a` to list all containers, including stopped ones.
- Use `docker logs <container_name>` to check the logs of the stopped container.

Use `docker inspect <container_name>` to get detailed information about the stopped container.

How to list all the image tags for a given container image?

To list all the tags for a given container image, you need to query the image registry that stores the image tags. For Docker Hub, this can be done via the Docker Hub API, but there isn't a built-in Docker CLI command to list image tags directly. Below are a few approaches to find the tags for a given container image:

1. Using Docker Hub API (for public images on Docker Hub)

If the image is hosted on Docker Hub (for example, the `nginx` image), you can use the Docker Hub API to retrieve all tags associated with a specific image.

Here is how you can do it:

1. API Request via `curl`:

You can use the `curl` command to fetch the tags for a given image.

```
curl -s https://registry.hub.docker.com/v1/repositories/<image_name>/tags | jq '.[].name'
```

- Replace `<image_name>` with the name of the image you're interested in (e.g., `nginx`).
- `jq` is used to parse the JSON output and show the tag names.

Example for `nginx`:

```
curl -s https://registry.hub.docker.com/v1/repositories/nginx/tags | jq '.[].name'
```

This will output a list of all tags associated with the `nginx` image on Docker Hub.

2. Example Output:

```
1.19.0
1.18.0
1.17.10
latest
```

2. Using Docker CLI (for locally available images)

If the image is available on your local machine, you can list all tags for images using the `docker images` or `docker image ls` command.

1. List all local images:

You can list all images on your local system using:

```
docker images
```

or

```
docker image ls
```

This command will show a list of images with their tags, repository names, image IDs, and sizes.

Example Output:

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
nginx	latest	d1d9f7f5b1ed	4 days ago	133MB
nginx	1.19.0	9a5a5a5072db	2 weeks ago	131MB
my_image	v1	7d9f2f3f4cfd	3 weeks ago	150MB

From this output, you can see all the tags for the `nginx` image (`latest`, `1.19.0`, etc.).

3. Using Docker Hub Web Interface

For public images hosted on Docker Hub:

1. Go to the Docker Hub website (<https://hub.docker.com/>).
2. Search for the image (e.g., `nginx`).
3. On the image page, you can see a list of tags under the "Tags" tab or section.

This is a manual method, but it's straightforward and doesn't require any command-line tools.

`docker search --list-tags IMAGE_NAME`

The command `docker search --list-tags IMAGE_NAME` does not exist in Docker's official command-line interface (CLI).

Explanation:

Docker's `docker search` command is used to search Docker Hub for images, but it doesn't support listing tags directly. If you use `docker search <image_name>`, it will show a list of repositories matching the image name, but not the tags for the image.

For example:

```
docker search nginx
```

This will give you a list of available repositories with the `nginx` name, but **not the tags**.

Correct Approach to List Tags for an Image

If you want to list the tags for a Docker image, you would need to use other methods, as I mentioned in the previous answer:

1. Using Docker Hub API (for public images)

You can use the `curl` command to query the Docker Hub API for tags:

```
curl -s https://registry.hub.docker.com/v1/repositories/<image_name>/tags | jq  
'.[].name'
```

Replace `<image_name>` with the desired image (e.g., `nginx`).

2. List Images Locally:

If you have the image downloaded locally, you can use the following to list images and their tags:

```
docker images
```

This will show the images and tags that are available on your local machine.

Unfortunately, Docker CLI doesn't have a direct command like `docker search --list-tags IMAGE_NAME` to list tags for a specific image. The best way to do this would be via Docker Hub's API or by using third-party tools.

Why container images are relatively small? You are interested in running a container with snake game application. How can you search for such image and check if it exists?

Why Container Images Are Relatively Small

Container images are generally smaller than traditional virtual machine (VM) images due to the following reasons:

1. **Minimal Operating System:**

Containers typically only contain the application and its dependencies, not a full OS. Unlike virtual machines, which include an entire guest OS, containers rely on the host system's OS kernel. This means that container images can be much smaller because they don't need to carry the entire OS stack.

2. **Layered File System:**

Container images use a layered architecture, where each layer represents a set of changes (such as adding files, installing libraries, etc.). Layers are cached and reused across different images, so the same base layers (e.g., operating system or application dependencies) don't have to be repeated in every image. This reduces duplication and saves space.

3. **Efficient Packaging:**

Containers are typically built with efficiency in mind. Only the necessary binaries, libraries, and configurations needed to run the application are included. Many container images are built from minimal base images (like `alpine` or `busybox`), which further reduces their size.

4. **No Overhead for Virtualization:**

Virtual machines require an entire OS, hypervisor, and management layer, while containers only run the application with the OS kernel shared from the host machine. As a result, the size of the container image is much smaller.

How to Search for a Snake Game Docker Image

To search for a container image with a specific application, like a Snake game, you can use the `docker search` command or directly search on Docker Hub.

1. Search Using Docker CLI:

To search for an image related to a "snake game" using Docker CLI:

```
docker search snake-game
```

This will return a list of Docker images matching the keyword "snake-game" from Docker Hub.

2. Search on Docker Hub Website:

You can also directly go to Docker Hub and search for "snake game" in the search bar. This will show you any available images related to the Snake game.

3. Checking for Existence and Pulling the Image:

If you find a relevant image after searching (for example, an image called `snake-game`), you can check if it exists on your local system using:

```
docker images
```

If it doesn't exist, you can pull it from Docker Hub by running:

```
docker pull <image_name>
```

For example, if the image is named `snake-game`, you would run:

```
docker pull snake-game
```

4. Running the Container:

Once you have the image, you can run the container using:

```
docker run -d --name snake-game-container <image_name>
```

For example:

```
docker run -d --name snake-game-container snake-game
```

This command will run the Snake game container in the background.

Conclusion

- **Container images** are small because they are minimal, based on layers, and do not include the full OS, unlike virtual machines.
- To **search for a snake game image**, you can use `docker search` or visit Docker Hub.

If the image exists, you can pull it and run it with Docker commands.

Where Pulled Images Are Stored

When you pull an image using Docker, it is stored locally on the host machine. By default, Docker stores images in a specific directory in its file system. The location of this directory depends on the operating system you're using:

- **Linux:** Images are stored in `/var/lib/docker/`.
- **Windows (with Docker Desktop):** Images are stored in `C:\ProgramData\DockerDesktop\` or in a Docker-managed virtual disk.
- **macOS (with Docker Desktop):** Images are stored in a virtual machine's disk image, typically within `/Users/<user>/Library/Containers/com.docker.docker/`.

Docker manages these images in a layered filesystem, and you can access them directly via the Docker CLI but not directly from the filesystem in the usual way, as Docker uses a special format to manage layers and metadata.

Explain Container Image Layers

Container images in Docker are composed of multiple layers. Each layer represents a change or addition to the filesystem of the container, such as installing a package, modifying a configuration file, or adding a file. These layers are stacked on top of one another to form the final image.

How Layers Work:

- **Layered Structure:** Docker uses a union filesystem to combine layers. Each layer is a snapshot of the filesystem at a specific point in time.
- **Immutable Layers:** Once a layer is created, it cannot be modified. Any change or update (e.g., installing a new package) will create a new layer on top.
- **Shared Layers:** If two or more images share common layers, Docker reuses these layers, which saves disk space and speeds up operations (like pulling or building images).

Example:

For an image like `ubuntu`, the base image (`ubuntu:latest`) may consist of multiple layers such as:

- A base layer with Ubuntu OS.
- A layer where `apt-get` is run to install dependencies.
- A layer where an app or service is added, such as `nginx` or `python`.

Why Layers Are Important:

- **Efficiency:** Layered images allow Docker to reuse layers across different images. This helps in reducing storage space and download time when multiple images share the same layers.
- **Caching:** Docker uses layers as cache, meaning if the content in a layer hasn't changed, Docker doesn't need to rebuild it when building new images.

True or False? Changing the Content of Any of the Image Layers Will Cause the Hash Content of the Image to Change

True:

If you change the content of any layer in an image, the hash (or checksum) of that layer will change. This is

because Docker generates a hash for each layer based on its content. Even a small change, such as modifying a single file in the layer, will cause the hash to change, and thus the resulting image's checksum will be different.

Since Docker calculates an image's hash by combining the hashes of each layer, any modification in one layer results in a new image with a different hash.

How to List the Layers of an Image

To list the layers of an image, you can use the `docker history` command. This will show you the layers that make up a Docker image, including details such as the commands used to create each layer, the size of each layer, and the date each layer was created.

Here's the command:

```
docker history <image_name>
```

For example:

```
docker history ubuntu
```

This command will list all the layers of the `ubuntu` image. The output will include:

- **Layer ID:** The unique identifier for the layer.
- **Created:** The time when the layer was created.
- **Size:** The size of the layer.
- **Command:** The command used to create that layer (e.g., `RUN apt-get install`).

Example Output of `docker history ubuntu`

IMAGE	CREATED	CREATED BY	SIZE
COMMENT			
<layer_id>	2 days ago	/bin/sh -c #(nop) CMD ["/bin/bash"]	0B
<layer_id>	2 days ago	/bin/sh -c apt-get update && apt-get install -y	75MB
<layer_id>	2 days ago	/bin/sh -c #(nop) LABEL org.opencontainers....	0B
<layer_id>	2 days ago	/bin/sh -c #(nop) MAINTAINER Ubuntu <ubuntu@...	0B
<layer_id>	2 days ago	/bin/sh -c #(nop) FROM ubuntu:20.04	0B

This command will display the layers for the specified image. Each layer in the output represents a step in the process of building the image.

✓ True or False? In most cases, container images contain their own kernel

✗ False

Container images **do not contain their own kernel**. Containers share the **host operating system's kernel**. This is one of the key differences between containers and virtual machines.

- **Containers** package up application code and its dependencies (binaries, libraries, etc.), but **rely on the host OS kernel** to run.
- This design makes containers lightweight and efficient but also ties them to the kernel features and capabilities of the host OS.

✓ True or False? A single container image can have multiple tags

✓ True

A **single container image** can have **multiple tags**. Tags are human-readable aliases (like `v1.0`, `stable`, `latest`) that reference the same image ID (SHA256 hash).

For example:

```
docker tag my-image:latest my-image:v1.0
```

This means the **same image** is now accessible using either `my-image:latest` or `my-image:v1.0`.

This is useful for:

- Version control
- Rollback strategies

Managing multiple release labels for the same image

What is a dangling image?

A **dangling image** is a Docker image that **has no tag** and is **not referenced by any container**. These are typically **intermediate images** created during a build process that are no longer needed.

You can identify dangling images with:

```
docker images -f "dangling=true"
```

They usually have:

- `<none>` as the **repository** and **tag**
- They still consume disk space and can be cleaned up

You can remove them using:

```
docker image prune
```

How to see changes done to a given image over time?

To see the **layer-by-layer changes** made to an image, you can use:

```
docker history <image-name>
```

Example:

```
docker history nginx
```

This will show each **instruction** (like `RUN`, `COPY`, etc.) used during the build, along with the **size**, **timestamp**, and **command** that created each layer.

Note: This doesn't show file-level differences, just the Dockerfile instruction history.

What does `docker commit` do? When would you use it?

The `docker commit` command creates a **new image from the current state** of a running or stopped container.

Syntax:

```
docker commit <container-id> <new-image-name>
```

Use Case:

- You manually make changes **inside a container** (e.g., install packages, edit files), and want to **capture that state** as a reusable image.

When to use:

- **For debugging or experimentation**, not for production builds.
- Use `Dockerfile` and `docker build` for reproducible, version-controlled builds.

Example:

1. Run a container and make some changes:

```
docker run -it ubuntu
apt update && apt install curl
```

2. Open a new terminal and commit the container:

```
docker ps # get container ID
docker commit <container-id> my-custom-ubuntu
```

3. Now you have a new image (`my-custom-ubuntu`) with `curl` installed.

True or False? Multiple images can share layers

✓True

Docker images are built in layers, and **multiple images can share common layers**. If two images have identical base layers (e.g., `ubuntu:20.04`), Docker stores those shared layers **only once** on disk.

✓ **Benefits:**

- Saves disk space
- Speeds up image builds and pulls

What is the digest of an image? What problem does it solve?

A **digest** is a **cryptographic hash (SHA256)** that uniquely identifies a specific version of an image, including **all of its layers and metadata**.

Example:

```
nginx@sha256:6b914bbcb...
```

What problem does it solve?

- Ensures **immutability and integrity**: unlike tags (which are mutable), a digest guarantees that you're pulling the **exact same image content** every time.
- Prevents issues where `latest` or a tag is changed upstream but you want **consistent deployments**.

True or False? A single image can support multiple architectures (Linux x64, Windows x64, ...)

✓True

A single image name can refer to a **multi-architecture (multi-arch) manifest**, which allows Docker to **automatically pull the correct image variant** based on the system architecture (e.g., `x86_64`, `ARM64`, `Windows`).

Examples of multi-arch images:

- `nginx`
- `alpine`
- `python`

These use a **manifest list** that maps to platform-specific images.

```
docker manifest inspect nginx
```

This feature is part of Docker's **OCI image specification** and helps support **cross-platform compatibility**.

What is a distribution hash in regards to layers?

A **distribution hash** is the **digest** (usually SHA-256) of a **compressed image layer or manifest file**.

Context:

In Docker/OCI (Open Container Initiative) images:

- Every **layer** (compressed tarball) and **configuration** file is content-addressed using a hash (usually `sha256:<hash>`).
- This hash is how registries, clients, and runtimes **identify and verify** the integrity of the content.

Example:

If you inspect an image with `docker image inspect`, you might see something like:

```
"RootFS": {
  "type": "layers",
  "diff_ids": [
    "sha256:abc123...",
    "sha256:def456..."
  ]
}
```

These are layer **hashes**—each representing a filesystem change. When pulling from a registry, you will see `sha256:<digest>` indicating the verified content being downloaded.

How do multi-architecture images work?

Multi-arch images are essentially **manifest lists** (also called **fat manifests**) that point to architecture-specific images.

How it works:

1. A **manifest list** (`index.json`) contains references to platform-specific manifests (e.g., for `amd64`, `arm64`, etc.).
2. Each of those platform-specific manifests then lists the layer blobs and config for that specific architecture.

Example:

```
docker manifest inspect --verbose ubuntu
```

This may show entries like:

```
{
  "platform": {
    "architecture": "amd64",
```

```
"os": "linux"
},
"digest": "sha256:abcd..."
}
```

So when a user on an `arm64` host pulls the image, the client (e.g., Docker) selects the correct image for that platform.

What happens when you pull a multi-arch image?

Let's say you're on an **ARM64** system and you run:

```
docker pull ubuntu
```

Here's what happens:

1. Docker client queries the registry for the manifest list for `ubuntu:latest`.
2. It receives a manifest list with multiple entries (for `amd64`, `arm64`, etc.).
3. Docker selects the matching architecture for your host (e.g., `arm64`).
4. Docker pulls the platform-specific manifest.
5. It then pulls all the layer blobs listed in the manifest.
6. These layers are cached locally and used to create containers.

How to check which architectures a certain container image supports?

You can use the `docker` or `skopeo` tools:

With Docker:

```
docker manifest inspect <image-name>
```

For example:

```
docker manifest inspect --verbose nginx
```

How to check what a container will execute when run from an image?

This is about checking the image's **entrypoint** and **command (CMD)**.

Using Docker:

```
docker inspect <image-name>
```

Look for fields:

- `"Entrypoint"`: this is the main executable.
- `"Cmd"`: default arguments if not overridden.

Example:

```
docker inspect ubuntu
```

Might show:

```
"Entrypoint": null,  
"Cmd": [  
  "bash"  
]
```

So running `docker run ubuntu` will execute `bash` by default.

✓ Summary:

Concept	How to Check / What It Means
Distribution hash	Content hash of layers or config files (<code>sha256:<digest></code>)
Multi-arch images	Use manifest lists to serve architecture-specific images
Pull process	Client resolves correct manifest for platform, downloads layers
Supported architectures	<code>docker manifest inspect <image></code> or <code>skopeo inspect docker://<image></code>
Default execution (Entrypoint)	<code>docker inspect <image></code> → Check Entrypoint and Cmd

What ways are there to reduce container image size?

Reducing image size improves **build speed**, **security**, and **deployment performance**.

✓ Common techniques:

Technique	Description
Use smaller base images	Use <code>alpine</code> , <code>debian-slim</code> , or <code>distroless</code> instead of full <code>ubuntu</code> or <code>debian</code>
Multi-stage builds	Build in one stage, copy only final artifacts to a minimal image
Remove package manager caches	E.g., <code>apt-get clean</code> , <code>rm -rf /var/lib/apt/lists/*</code> in Debian-based distros
Combine RUN statements	Reduces the number of layers and metadata overhead
Avoid unnecessary files	Use <code>.dockerignore</code> to exclude build files like <code>node_modules</code> , <code>.git</code> , <code>logs</code>
Use tools like <code>docker-slim</code>	Automatically analyze and shrink images by removing unused components
Minimize installed dependencies	Only install what's strictly required (e.g., no compilers in final image)

What are the pros and cons of squashing images?

Squashing means combining multiple layers into one to reduce image size and layer complexity.

✓Pros:

- **Smaller image size** (sometimes).
- **Fewer layers** – can simplify image for transport or storage.
- Better for **security audits** (fewer layers to inspect).

✗Cons:

- **Breaks layer cache reuse** – modifying one part might force you to rebuild everything.
- Harder to **trace changes** per Dockerfile instruction.
- If you squash everything into one layer, **you lose intermediate layer history**.

You can squash with:

```
docker build --squash .
```

Or post-build with `buildkit` or image tools like `img`, `buildah`.

How to share an image with another developer without using a registry?

You can export and import images **locally** without Docker Hub or any registry.

✓ Option 1: `docker save` / `docker load`

Export:

```
docker save -o my-image.tar <image-name>
```

Send `my-image.tar` via USB, SSH, email, etc.

Import on the other machine:

```
docker load -i my-image.tar
```

✓ Option 2: Use `docker export` / `docker import`

This is **not recommended** for preserving image metadata, but useful in minimal cases.

Export a container filesystem:

```
docker export <container-id> > container.tar
```

Import as image:

```
docker import container.tar my-new-image
```

⚠ Note: This doesn't preserve history, layers, or Dockerfile metadata.

What is a Dockerfile?

- A **Dockerfile** is a plain text file that contains a set of instructions to build a Docker image.
- It automates the process of creating an image by defining the **base image**, commands to install software, copy files, set environment variables, and specify the default command to run.

What instruction exists in every Dockerfile and what does it do?

- The **FROM** instruction exists in every Dockerfile (except some special cases).
- **Purpose:** Specifies the **base image** to build upon.

Example:

```
FROM ubuntu:20.04
```

This tells Docker to start the build from the Ubuntu 20.04 image.

List five different instructions available in a Dockerfile

Instruction	Purpose	Example
RUN	Execute commands during build (install packages, etc.)	RUN apt-get update && apt-get install -y curl
COPY	Copy files/directories from build context into the image	COPY ./app /app
ENV	Set environment variables	ENV NODE_ENV=production
CMD	Set default command to run when container starts	CMD ["nginx", "-g", "daemon off;"]
EXPOSE	Document which ports the container listens on	EXPOSE 80

What are some best practices regarding Dockerfiles?

- Use **small and official base images** (e.g., alpine, debian-slim).
- Use **multi-stage builds** to keep images small by copying only final artifacts.
- Combine multiple **RUN** commands into a single one to reduce layers.
- Use **.dockerignore** to exclude unnecessary files from the build context.
- Pin versions for dependencies and base images to ensure reproducibility.
- Avoid storing secrets or sensitive data in the Dockerfile.
- Set a **non-root user** to improve security.

What is the "build context"?

- The **build context** is the set of files and directories that Docker has access to when building an image.
- Usually, it is the directory you specify in the `docker build` command (e.g., `docker build .` uses the current directory as the build context).
- Docker sends this context (including the Dockerfile and files to COPY) to the Docker daemon during the build.

Large or unnecessary files in the build context slow down builds—use `.dockerignore` to exclude them.

Difference between ADD and COPY in a Dockerfile

Feature	COPY	ADD
Basic use	Copies files and directories from build context into the image	Same as COPY, but with extra features
Supports remote URLs	No	Yes (can download and extract files from URLs)
Supports tar extraction	No	Yes (if the source is a local tar archive, it will auto-extract)
Recommended usage	For simple, straightforward file copying	For when you need to unpack archives or fetch remote files

Best practice:

Use `COPY` unless you explicitly need the extra functionality of `ADD` (like auto-extracting archives or downloading URLs). This makes your Dockerfile more predictable.

Difference between CMD and RUN in a Dockerfile

Instruction	Purpose	When executed
RUN	Executes commands during image build (e.g., install software)	At build time; creates a new layer in the image
CMD	Specifies the default command to run when a container starts	At container runtime; can be overridden by user

Examples:

```
RUN apt-get update && apt-get install -y curl
# This installs curl while building the image
```

```
CMD ["nginx", "-g", "daemon off;"]
# This runs nginx when the container starts
```

Summary:

Instruction	When it runs	Purpose
RUN	During build	Modify image (install packages, run scripts)
CMD	At container start	Define default executable/command for container

Dockerfile instructions that create new layers

Each of these instructions creates a **new image layer**:

- **RUN** — executes commands and commits the results as a new layer
- **COPY** — copies files/directories into the image
- **ADD** — copies files/directories or remote URLs into the image (with some extras)
- **CMD** (technically metadata, but included in image config) — *does NOT* create a new layer (see below)
- **ENTRYPOINT** (metadata only, no new layer)
- **ENV** (metadata only, no new layer)
- **WORKDIR** (metadata only, no new layer)
- **USER** (metadata only, no new layer)
- **EXPOSE** (metadata only, no new layer)
- **VOLUME** (metadata only, no new layer)
- **LABEL** (metadata only, no new layer)
- **ARG** (build-time variable, no new layer)

So, strictly speaking, the instructions that **create new filesystem layers** are mainly:

Instructions creating new layers
RUN
COPY
ADD

Dockerfile instructions that create image metadata (do NOT create new layers)

The following instructions **do not create a new filesystem layer**, but instead add metadata to the image configuration:

Instructions that add metadata only

CMD

ENTRYPOINT

ENV

Instructions that add metadata only

WORKDIR

USER

EXPOSE

VOLUME

LABEL

ARG

Summary Table

Instruction	Creates New Layer?	Purpose
RUN	Yes	Run command, modify filesystem
COPY	Yes	Copy files
ADD	Yes	Copy files + extras
CMD	No	Default container command
ENTRYPOINT	No	Container startup command
ENV	No	Environment variables
WORKDIR	No	Set working directory
USER	No	Set user for container
EXPOSE	No	Document exposed ports
VOLUME	No	Declare volumes
LABEL	No	Metadata labels
ARG	No	Build-time variables

What's the difference between these two forms:

```
ENTRYPOINT ["cmd", "param0", "param1"]  
CMD ["param0"]
```

```
ENTRYPOINT cmd param0 param1  
CMD param0
```

This is about **exec form** vs **shell form** of `ENTRYPOINT` and `CMD` in Dockerfiles, and how they behave differently.

1. Exec form (JSON array)

```
ENTRYPOINT ["cmd", "param0", "param1"]  
CMD ["param0"]
```

- **Exec form** is the preferred form.
- The command and arguments are passed **directly** to the kernel (no shell involved).
- Each argument is a separate JSON string element.
- **No shell processing**, so no variable expansion, globbing, or word splitting.
- This means signals (like `SIGTERM`) are properly forwarded to the process.

How `CMD` and `ENTRYPOINT` interact here:

- `ENTRYPOINT` is fixed as `cmd param0 param1`.
- `CMD ["param0"]` provides **additional arguments** appended to the `ENTRYPOINT`.
- Resulting command run is:

```
cmd param0 param1 param0
```

2. Shell form (string)

```
ENTRYPOINT cmd param0 param1  
CMD param0
```

- Shell form is a **single string** executed via the shell (usually `/bin/sh -c`).
- This allows shell features like variable expansion, chaining, pipes.
- However, the actual process is the shell, not your command — signal handling is less direct.

How `CMD` and `ENTRYPOINT` interact here:

- `ENTRYPOINT` runs as:

```
/bin/sh -c "cmd param0 param1"
```

- `CMD` is ignored if an `ENTRYPOINT` is specified in shell form, **unless** the container is started with extra command arguments.

Summary

Aspect	Exec Form (<code>["cmd", "arg1"]</code>)	Shell Form (<code>cmd arg1</code>)
Command execution	Direct exec of command, no shell	Runs command inside <code>/bin/sh -c shell</code>
Signal handling	Proper signal forwarding	Signals go to shell, which may not forward properly
Argument separation	Each argument separate, no word splitting	Single string, subject to shell parsing
Interaction of CMD & ENTRYPOINT	CMD adds arguments to ENTRYPOINT	CMD often ignored if ENTRYPOINT set in shell form
Recommended usage	Yes, especially for ENTRYPOINT to avoid shell issues	Useful for simple commands needing shell features

Example:

```
ENTRYPOINT ["echo", "Hello"]  
CMD ["World"]
```

Running container prints:

```
Hello World
```

But:

```
ENTRYPOINT echo Hello  
CMD World
```

Runs:

```
/bin/sh -c "echo Hello"
```

and ignores CMD.

Persistent Storage Options

1. Volumes (Managed by Docker)

- **Best option** for persistent data.
- Created and managed by Docker itself.
- Data is stored in `/var/lib/docker/volumes/...` by default (host path may vary).
- Volumes are independent of containers and survive container deletion.

Create a volume:

```
docker volume create mydata
```

Use a volume:

```
docker run -v mydata:/app/data myimage
```

✓ **Best for:** databases, user data, app state.

2. Bind Mounts (Host filesystem paths)

- Mounts a **specific directory or file** from the host into the container.
- More flexible and transparent than volumes.
- Changes on the host are visible in the container and vice versa.

Example:

```
docker run -v /host/path:/app/data myimage
```

✓ **Best for:** local development, sharing config files, debugging.

⚠ **Be careful:** changes to host paths can affect container behavior.

3. tmpfs Mounts (In-memory)

- Temporary filesystem in RAM.
- **Data does not persist** even if the container keeps running long-term.
- Useful for sensitive data (e.g., credentials) or fast temp file access.

Example:

```
docker run --tmpfs /app/tmp myimage
```

✓ **Best for:** non-persistent scratch space, secrets, or caching.

Summary Table

Storage Type	Persistent?	Managed by Docker?	Use Case
Volume	✓Yes	✓Yes	Long-term app data, DB storage
Bind Mount	✓Yes	✗No (host-managed)	Local dev, config sharing
tmpfs	✗No	✗No	In-memory temp storage

Best Practices

- Use **volumes** for production apps requiring persistent data.
- Use **bind mounts** for development or when you need direct access to host files.
- Avoid writing persistent data to the container filesystem (/) directly.
- Backup and monitor **volumes** regularly in production.

How do containers achieve isolation from the rest of the system?

Containers use **Linux kernel features** to isolate their processes, filesystems, networking, and resource usage from the host system and other containers — similar to lightweight virtual machines, but without the overhead.

Key Linux Kernel Features Used by Containers

Feature	Description
Namespaces	Provide isolation for processes, filesystems, networking, user IDs , and more. Each container gets its own "view" of the system.
cgroups (control groups)	Control and limit container resource usage — CPU, memory, I/O, etc.
Union Filesystems	Combine read-only image layers with a writable container layer (e.g., OverlayFS).
Capabilities	Allow fine-grained control over what privileged operations a containerized process can perform.
seccomp	Filters syscalls available to a container for added security.
AppArmor / SELinux	Apply mandatory access control (MAC) policies to restrict what containers can access.

Describe difference between cgroups and namespaces

While both **cgroups** and **namespaces** are foundational to container technology, they serve **different purposes** in isolation and control.

1. Namespaces: "What a process can see"

Namespaces isolate **system resources** for processes. They create a limited view of the system, so each container thinks it's running on its own machine.

Types of Namespaces and What They Isolate:

Namespace	Isolates...
pid	Process IDs (each container has its own PID 1)
mnt	Mount points (separate filesystem)
net	Network interfaces, IPs, ports
uts	Hostname and domain name
ipc	System V and POSIX message queues
user	User and group IDs (UID/GID mapping)
cgroup	View of control groups

✓ Example:

- Container A's process list only shows its own processes, not those from Container B or the host.
- Each container can have its own `hostname`.

2. cgroups: "How much a process can use"

Control groups (cgroups) manage and limit **resource usage** — they don't isolate, they **restrict**.

cgroups can control:

Resource	Control Type
CPU	Limit CPU time
Memory	Set memory usage limits
Disk I/O	Throttle read/write speed
Network (v2)	Limit bandwidth
Process count	Limit number of forked processes

Resource	Control Type
Devices	Restrict access to hardware devices

✓Example:

- A container is limited to 512 MB of memory and 1 CPU core.
- If it exceeds the memory, it may be killed by the kernel (OOM).

Summary: cgroups vs namespaces

Feature	Namespaces	cgroups
Purpose	Isolation (what a container can see)	Resource control (what it can use)
Type	Access-level sandbox	Usage-level control
Key function	Hides host details, creates isolated views	Limits memory, CPU, disk, etc.
Used for	Process, network, filesystem separation	Preventing resource exhaustion

They **work together** to create the secure and efficient isolation model that containers rely on. Think of it as:

- Namespaces = *"your own private world"*
- cgroups = *"stay within your lane"*

Which components/layers compose Docker technology?

Docker is composed of several **layered components** that work together to build, run, and manage containers:

Docker Components:

Layer	Description
Docker CLI	The command-line interface (<code>docker</code>) used by developers/admins.
Docker Engine	The core client-server app that runs containers.
Docker Daemon (<code>dockerd</code>)	Manages images, containers, volumes, networks. Runs in the background.
Container Runtime	Runs and manages containers using Linux kernel features.
Image Format	The blueprint for containers (layers, metadata, configs).
Docker Hub / Registry	Stores and distributes images.

What components are part of the Docker Engine?

The **Docker Engine** is the core of Docker's container platform. It includes:

Main Components:

1. **Docker Daemon (dockerd)**
 - Listens for Docker API requests
 - Manages containers, images, volumes, networks
 - Spawns containers using container runtimes
2. **Docker CLI (docker)**
 - Command-line client that interacts with the Docker daemon via REST API
3. **REST API**
 - Programmatic interface used by CLI or other tools to control Docker
4. **Container Runtime (like runc)**
 - Low-level tool that actually creates and manages container processes

What is the low-level runtime?

Low-level Runtime = `runc`

- A lightweight, standalone binary that creates and runs containers.
- **Implements the OCI (Open Container Initiative) runtime specification.**
- Used by Docker to actually launch and manage containers using Linux features like namespaces and cgroups.

Think of `runc` as the "engine that runs a single container."

What is the high-level runtime?

High-level Runtime = Container Manager (e.g., `containerd`)

- A long-running **daemon** that manages the **lifecycle of containers**.
- Handles image pulls, storage, networking, and uses `runc` to create containers.
- Acts as an **abstraction layer** between Docker and low-level runtimes.

Think of `containerd` as the "manager" that talks to `runc` to run containers.

Diagram Overview

```
[ Docker CLI ]
    ↓
[ Docker Daemon (dockerd) ]
    ↓
[ containerd (high-level runtime) ]
    ↓
[ runc (low-level runtime) ]
    ↓
[ Linux Kernel (namespaces, cgroups, etc.) ]
```

✓ Summary

Layer	Example	Role
High-level runtime	containerd	Manages lifecycle of containers
Low-level runtime	runc	Actually runs containers (OCI)
Docker Engine components	CLI, Daemon, API	Interface + core container management
Kernel features	namespaces, cgroups	Provides isolation and resource limits

What happens when you run `docker pull image:tag`?

When you run:

```
docker pull image:tag
```

Step-by-step detailed process:

1. **Docker CLI sends a pull request to the Docker daemon (dockerd).**
2. **Daemon checks if the image:tag is already present locally.**
 - If yes, it skips downloading.
 - If no, it proceeds to download.
3. **Daemon contacts the image registry (usually Docker Hub unless configured otherwise).**
4. **Registry authentication (if needed):**
 - Docker daemon authenticates using credentials stored locally (~/.docker/config.json).
5. **Image manifest retrieval:**
 - Daemon fetches the **manifest** describing the image's layers and metadata.
6. **Layer-by-layer download:**
 - The image is composed of multiple layers (filesystem diffs).
 - Docker checks which layers are already downloaded locally.
 - Only missing layers are downloaded.
7. **Layer verification:**
 - Each downloaded layer is verified with a digest checksum for integrity.
8. **Layers are stored locally in Docker's storage area (/var/lib/docker).**
9. **Image metadata is updated to include the new image:tag.**
10. **Completion:**
 - Once all layers are downloaded and verified, the image:tag is ready to be used locally.

What happens when you run a container?

When you run:

```
docker run image:tag [command]
```

Step-by-step detailed process:

1. **Docker CLI sends a container creation and run request to the Docker daemon (dockerd).**
2. **Daemon verifies the image exists locally:**
 - If not, triggers `docker pull` automatically.
3. **Container creation:**
 - Creates a **read-write container layer** on top of the read-only image layers.
 - Allocates namespaces for isolation (PID, network, mount, etc.).
 - Sets up cgroups to limit resources.
 - Prepares the container's network stack and mounts.
4. **Configures the container environment:**
 - Sets environment variables.
 - Applies ENTRYPOINT and CMD instructions.
 - Configures volumes and bind mounts if specified.
5. **Starts the container process:**
 - Invokes the low-level runtime (`runc`) to launch the containerized process inside the isolated namespaces and cgroups.
6. **Containerized application runs:**
 - Output is streamed back to the CLI if attached.
 - Container runs in foreground or background (depending on `-d`).
7. **Lifecycle management:**

Container keeps running until the process exits or is stopped.

What is Docker Compose?

- **Docker Compose** is a tool for defining and running **multi-container Docker applications**.
- Instead of running many `docker run` commands, you describe your entire app's services, networks, and volumes in a single **YAML file** called `docker-compose.yml`.
- It orchestrates how containers interact, making it easy to spin up or tear down complex environments with one command.

What is Docker Compose used for?

- **Define multi-container apps** (web server + database + cache + etc.) declaratively.
- Manage **service dependencies** and order of startup.
- Simplify **environment setup** for development, testing, and staging.
- Easily scale services (`docker-compose up --scale`).
- Share reproducible environment definitions across teams.

Describe the process of using Docker Compose

Step 1: Write a `docker-compose.yml` file

- Define your services with their images, build context, ports, environment variables, volumes, networks, and other configs.

Example:

```
version: '3'
services:
  web:
    image: nginx:latest
    ports:
      - "80:80"
  db:
    image: postgres:13
    environment:
      POSTGRES_PASSWORD: example
    volumes:
      - pgdata:/var/lib/postgresql/data
volumes:
  pgdata:
```

Step 2: Start the application stack

Run:

```
docker-compose up
```

- This creates the network, volumes, and starts containers as defined.
- Add `-d` to run in detached mode (`docker-compose up -d`).

Step 3: Manage your application

- **Stop services:**

```
docker-compose down
```

Stops and removes containers, networks, and by default volumes.

- **View logs:**

```
docker-compose logs -f
```

- **Scale services:**

```
docker-compose up --scale web=3
```

- **Rebuild services:**

```
docker-compose build
```

Step 4: Share your setup

- Share the `docker-compose.yml` file with your team.
- Everyone can spin up the exact same environment with `docker-compose up`.

Summary:

Step	Action	Purpose
1. Write YAML	Define services, networks, volumes	Describe app architecture
2. <code>docker-compose up</code>	Launch multi-container app	Start all services
3. Manage	Stop, scale, logs, rebuild	Control app lifecycle
4. Share	Share YAML file	Reproducibility & collaboration

What are Multi-stage Builds?

- **Multi-stage builds** allow you to use **multiple FROM statements** in a single Dockerfile.
- Each FROM starts a new build stage.
- This technique helps you **separate the build environment from the final runtime image**, reducing the final image size.
- You can build your app and its dependencies in one stage, then copy only the necessary artifacts (binaries, files) into a clean, minimal base image in a later stage.
- This avoids shipping unnecessary build tools or intermediate files into your production image.

Example of a multi-stage build:

```
# Stage 1: Build
FROM golang:1.20 AS builder
WORKDIR /app
COPY . .
RUN go build -o myapp

# Stage 2: Runtime
FROM alpine:latest
COPY --from=builder /app/myapp /usr/local/bin/myapp
CMD ["myapp"]
```

- First stage (builder) compiles the app.

Second stage copies only the compiled binary into a small Alpine image.

What is .dockerignore used for?

- The `.dockerignore` file tells Docker **which files and directories to exclude** from the **build context** when you run `docker build`.
- It works similarly to `.gitignore`, specifying patterns to **ignore**.
- This reduces the **build context size**, which:
 - Speeds up the build process (less data sent to the Docker daemon).
 - Avoids accidentally including sensitive or unnecessary files (e.g., `.git`, local config files, build artifacts).

Why is this important?

- Docker sends the entire build context (the directory you specify) to the daemon.
- If large or irrelevant files are included, builds become slower and bulkier.
- `.dockerignore` helps keep builds clean and efficient.

Example `.dockerignore` file:

```
node_modules
.git
*.log
temp/
.DS_Store
```

What container network standards or architectures are you familiar with?

1. Docker Default Networks

- **Bridge Network (default)**
 - Containers get connected to a virtual bridge (`docker0`), allowing them to communicate by IP or name.
 - Containers on the same bridge can communicate directly.
- **Host Network**
 - Container shares the host's network stack.
 - No network isolation; container uses host's IP address.
- **None Network**
 - No network interfaces attached.
 - Useful for isolated containers.

2. Container Network Interface (CNI)

- **CNI** is a standard for configuring networking for Linux containers.
- Used by Kubernetes, OpenShift, and other orchestrators.
- Provides plugins for different networking models:
 - **bridge**
 - **macvlan**
 - **flannel**
 - **calico**
 - **weave**
- Allows flexible, pluggable network setups.

3. Overlay Networks

- Virtual networks that span multiple Docker hosts or nodes.
- Used for multi-host container communication (e.g., Docker Swarm, Kubernetes).
- Encapsulates container traffic in VXLAN or other tunneling protocols.
- Enables containers across different physical hosts to communicate securely.

4. macvlan Networks

- Assigns a MAC address to each container.
- Containers appear as physical devices on the network.
- Useful for legacy apps requiring direct L2 network access.

5. User-defined Bridge Networks

- Custom bridges created by users.
- Supports container name resolution (DNS), network segmentation, and isolation.
- Preferred over default bridge for better control.

6. Service Mesh Networking

- Technologies like Istio or Linkerd run alongside containers.
- Handle advanced networking: service discovery, load balancing, encryption, observability.

Summary Table

Network Type	Use Case	Key Features
Bridge (default)	Single-host container comms	Isolation, container DNS
Host	High performance, no isolation	Shares host network stack
None	Isolated containers	No network interface
Overlay	Multi-host communication	Encrypted, tunneled networks
macvlan	Direct L2 access	Container has its own MAC/IP
CNI	Kubernetes and other orchestrators	Pluggable networking model
User-defined Bridge	Custom single-host networks	Custom DNS, segmentation
Service Mesh	Advanced microservices networking	Security, observability, retries

Security Best Practices for Containers

1. Run Containers as Non-root User

Avoid running containers as the root user inside the container. Use `USER` directive in Dockerfile to specify a non-root user.

2. Limit Container Capabilities

Drop unnecessary Linux capabilities with `--cap-drop` and add only what's needed with `--cap-add`.

3. Use Read-Only Filesystems

Mount container filesystems as read-only when possible (`--read-only` flag).

4. Use Minimal Base Images

Use slim or minimal images (like `alpine`) to reduce attack surface.

5. Scan Images for Vulnerabilities

Regularly scan container images for known vulnerabilities using tools like Trivy, Clair, or Docker Scan.

6. Limit Resource Usage

Use cgroups to limit CPU, memory, and I/O usage (`--memory`, `--cpus`).

7. Use User Namespaces

Map container users to non-root users on the host to reduce privilege escalation risks.

8. Apply Network Policies and Segmentation

Use network segmentation to isolate containers and restrict traffic flows.

9. Keep Host and Docker Engine Updated

Apply security patches regularly.

10. Avoid Privileged Containers

Do **not** use `--privileged` unless absolutely necessary.

11. Use Security Profiles

Leverage seccomp, AppArmor, or SELinux profiles to restrict syscalls and actions.

12. Limit Volume Mounts

Avoid mounting sensitive host directories into containers.

Preventing Kernel Panic Caused by Containers

Why can a container cause a kernel panic?

- Containers share the host kernel.
- Malicious or buggy containers can trigger kernel bugs, exploit vulnerabilities, or misuse kernel resources, causing kernel panic and host crash.

Preventive Actions:

1. Use Linux Security Modules (LSMs)

- **AppArmor** or **SELinux**: enforce mandatory access controls restricting container actions on the kernel and host.
- Apply strict policies to containers to prevent dangerous system calls or file access.

2. Apply Seccomp Profiles

- Limit the system calls containers can make.
- Block dangerous syscalls that could destabilize the kernel.
- Docker provides a default seccomp profile that blocks risky syscalls.

3. Run Containers with Limited Capabilities

- Drop unnecessary kernel capabilities to reduce attack surface.

4. Run Containers as Non-Privileged Users

- Avoid privileged containers which have broad kernel access.

5. Use User Namespaces

- Remap container root to an unprivileged user on the host.

6. Limit Resource Usage

- Prevent containers from exhausting kernel resources by limiting CPU, memory, PIDs (`--pids-limit`), and I/O.

7. Keep Kernel and Container Runtime Up to Date

- Apply patches to fix kernel vulnerabilities promptly.

8. Run Containers in Isolated Environments

- Use VM-based isolation (e.g., Kata Containers, Firecracker) for higher security boundaries if kernel exploits are a concern.

Summary:

Risk	Mitigation
Container exploits kernel bugs	Seccomp, LSMs (AppArmor/SELinux), drop caps
Excessive resource consumption	Cgroups limits (<code>--memory</code> , <code>--pids-limit</code>)
Privileged access to kernel	Avoid privileged mode, use user namespaces
Kernel vulnerabilities	Keep kernel updated, consider VM-based isolation

Example: Simple Seccomp Profile

Docker provides a **default seccomp profile** that blocks dangerous syscalls. Here's a minimal custom seccomp profile example that allows common syscalls but blocks `ptrace` (used for debugging, can be risky):

```
{
  "defaultAction": "SCMP_ACT_ERRNO",
  "archMap": [
    {
      "architecture": "SCMP_ARCH_X86_64",
      "subArchitectures": []
    }
  ],
  "syscalls": [
    {
      "names": [
        "read",
        "write",
        "exit",
        "exit_group",
        "rt_sigreturn",
        "sigreturn",
        "brk",
        "mmap",
        "munmap",
        "futex",
        "clone",
        "execve",

```

```

        "open",
        "close"
    ],
    "action": "SCMP_ACT_ALLOW"
}
]
}

```

- **defaultAction:** Blocks all syscalls not explicitly allowed.
- Only a small set of syscalls are allowed here.
- Save as `seccomp.json` and run container with:

```
docker run --security-opt seccomp=seccomp.json your_image
```

Example: Simple AppArmor Profile

Here's a minimal AppArmor profile that allows basic container operations but restricts access to certain system files:

```

profile my-container-profile flags=(attach_disconnected,mediate_deleted) {
    # Include common abstractions
    #include <abstractions/base>

    # Deny access to sensitive files
    deny /etc/shadow r,
    deny /etc/passwd w,

    # Allow read/write to container filesystem
    file,
    capability,

    # Network access
    network,

    # Allow all capabilities for demo (tighten this in prod)
    capability,

    # Deny ptrace syscalls
    deny ptrace,

    # Allow everything else by default
    audit deny /** wlx,
}

```

- Save it as `/etc/apparmor.d/my-container-profile`
- Load the profile with:

```
sudo apparmor_parser -r /etc/apparmor.d/my-container-profile
```

- Run container with:

```
docker run --security-opt apparmor=my-container-profile your_image
```

Notes:

- These examples are minimal and for demonstration only.
- In production, profiles should be **tuned carefully** to your container's needs.
- Docker's **default seccomp profile** is a great starting point and already blocks many risky syscalls.
- Tools like **aa-genprof** (for AppArmor) can help generate profiles based on container behavior.

Best Practices for Using Containers in Production

1. Use Minimal, Secure Base Images

Prefer small, well-maintained images like `alpine` or official images with minimal attack surface.

2. Run Containers as Non-root

Avoid running processes as root inside containers to reduce privilege escalation risks.

3. Apply Resource Limits

Use CPU, memory, and I/O limits (`--memory`, `--cpus`, `--pids-limit`) to prevent resource exhaustion.

4. Use Immutable Infrastructure Concepts

Treat containers as immutable units—replace instead of patch or update.

5. Enable Logging and Monitoring

Collect logs and metrics for containers to track performance and troubleshoot issues.

6. Use Orchestration Tools

Leverage Kubernetes, Docker Swarm, or other orchestrators for scaling, self-healing, and management.

7. Implement Network Segmentation and Security Policies

Isolate container networks and restrict traffic as needed.

8. Keep Host and Docker Updated

Regularly apply security patches to Docker Engine and underlying OS.

9. Scan Images for Vulnerabilities

Use tools like Trivy, Clair, or Docker Scan before deploying images.

10. Use Secrets Management

Avoid baking secrets into images—use secret management solutions or environment variables with care.

11. Use Signed Images and Trusted Registries

Verify image integrity and provenance.

Automate CI/CD Pipelines

Automate building, testing, and deploying container images with quality gates

Docker Restart Policies

When running a container, you specify restart policies with the `--restart` flag:

1. no (default)

- Do **not** restart the container automatically.
- The container stays stopped after exit.

2. always

- Always restart the container **regardless** of the exit status.
- If the container stops, Docker will restart it.
- If Docker daemon restarts, containers with this policy will also restart.

3. on-failure[:max-retries]

- Restart container **only if it exits with a non-zero exit code** (indicating failure).
- Optional `max-retries` limits how many times Docker attempts restart before giving up.
- If the container exits with code 0, it will **not** restart.

4. unless-stopped

- Similar to `always`, but **does not restart** containers when they are manually stopped.
- If the Docker daemon restarts, these containers will restart unless they were stopped by the user.

Summary Table

Restart Policy	When container restarts	Restart after daemon restart?	Stops on manual stop?
no (default)	Never	No	N/A
always	Always (any exit code)	Yes	No
on-failure[:n]	Only on non-zero exit code, up to n retries (optional)	Yes	No
unless-stopped	Always, except when manually stopped	Yes	Yes

Example Usage

```
docker run --restart=on-failure:5 my_app
```

- Restarts `my_app` container up to 5 times if it exits with failure.

```
docker run --restart=unless-stopped my_app
```

- Restarts unless container was manually stopped.

Summary

- Docker's **restart policies** are the primary built-in **self-healing** method.
- For more advanced self-healing, orchestration platforms (Swarm, Kubernetes) are recommended.

Using `HEALTHCHECK` instructions allows better detection of unhealthy containers for automatic remediation.

What are Rootless Containers?

- **Rootless containers** are containers run **without requiring root (administrator) privileges on the host system**.
- They allow **unprivileged users** to create and manage containers without giving them elevated access to the host.
- This is achieved by using **user namespaces**, where container root user is mapped to an unprivileged host user.
- Rootless mode isolates the container runtime and processes from host root privileges, reducing attack surface.

Disadvantages of Rootless Containers

1. **Limited Feature Set**
Some Docker features (like certain network drivers, port binding to low ports, or using `--privileged`) are **not fully supported** in rootless mode.
2. **Performance Overhead**
Rootless containers may have **slightly higher latency** due to user namespace translation and additional layers.
3. **Complex Setup**
Configuring rootless containers may require additional setup (e.g., user namespaces enabled, available subuids/subgids).
4. **Network Restrictions**
Binding to privileged ports (<1024) or certain network configurations may require workarounds.
5. **Limited Access to Host Resources**
Rootless containers can't directly manage or interact with some host resources that require root.

Security Advantage Example of Rootless Containers

- **Containment of Privilege Escalation**
Since the container processes run as **non-root users on the host**, even if an attacker breaks out of the container, they **won't gain root privileges on the host**. This limits the scope and severity of a potential container escape exploit.

Aspect	Description
Rootless containers	Containers run without root privileges on the host
Disadvantages	Limited features, network restrictions, complex setup
Security benefit	Reduces risk of host-level privilege escalation

OCI stands for Open Container Initiative.

- It is an open governance project under the Linux Foundation that defines **standards for container formats and runtimes**.
- OCI aims to ensure **interoperability and portability** of container images and runtimes across different container platforms.
- The two main specifications OCI defines are:
 - **OCI Image Specification:** Defines how container images are structured and stored.
 - **OCI Runtime Specification:** Defines how to run containers based on those images.

OCI-compliant container runtimes and tools must support operations that cover the **lifecycle of containers**, including:

1. **Image Operations:**
 - Pulling, storing, unpacking OCI-compliant images.
 - Supporting the OCI image layout and manifest formats.
2. **Container Lifecycle:**
 - Creating containers from OCI images.
 - Starting containers.
 - Stopping containers.
 - Pausing and resuming containers (if supported).
 - Deleting containers.
3. **Filesystem and Storage:**
 - Setting up container filesystems with layered image support.
 - Applying mounts and bind mounts as specified.
4. **Process and Namespace Management:**
 - Creating container process with appropriate Linux namespaces (PID, network, mount, user, etc.).
 - Managing cgroups for resource control.
5. **Standard I/O and Logging:**
 - Handling container standard input, output, and error streams.
 - Supporting logging mechanisms.
6. **Runtime Hooks:**
 - Support for pre-start, post-start, pre-stop hooks for additional lifecycle control.
7. **Configuration Compliance:**
 - Respecting OCI runtime configuration specs such as security profiles, capabilities, environment variables, user and group IDs.

Summary

OCI Aspect	What Must Be Supported
Image Spec	OCI image format compliance
Runtime Spec	Container lifecycle: create, start, stop, delete
Filesystem	Mount layered images and filesystems
Namespaces & cgroups	Setup container isolation & resource limits
I/O	Properly attach stdio and logging
Hooks	Lifecycle hooks support